

Lab 03: Real-World Data Linear Regression

Fitting a least-squares line to real population data with Python

Part A — Data

Source: World Bank, World Development Indicators (series SP.POP.TOTL, “Population, total”).

URL: <https://databank.worldbank.org/source/world-development-indicators> (cross-listed on FRED, Federal Reserve Bank of St. Louis: <https://fred.stlouisfed.org/series/POPTOTPHA647NWDB>)

Description: Midyear total population estimates for the Philippines, published annually by the World Bank based on national census and UN population data. The series is a de facto population count (all residents regardless of legal status), reported yearly from 1960 to the present.

Independent variable x: Year (calendar year, AD)

Dependent variable y: Total population of the Philippines (persons)

18 paired observations (2005–2022, one per year) were used, satisfying the minimum of 15 required.

Year (x)	Population, persons (y)	Year (x)	Population, persons (y)
2005	88,015,962	2014	103,767,130
2006	89,508,986	2015	105,312,992
2007	91,075,184	2016	106,735,719
2008	92,699,095	2017	108,119,693
2009	94,384,250	2018	109,465,287
2010	96,337,125	2019	110,804,683
2011	98,248,614	2020	112,081,264
2012	100,175,512	2021	113,100,950
2013	102,076,336	2022	113,964,338

Table A1. Philippines total population, 2005–2022.

Part B — Python Script

The complete, commented script below (also submitted separately as **regression.py**) loads the dataset, computes a_0 and a_1 with the least-squares formulas from class, reports S_r , r^2 , and $s_{y/x}$, and generates the fitted-line and residual plots.

```
"""
Lab 03: Real-World Data Linear Regression
Numerical Methods, Section 3D

Dataset : Philippines total population, 2005-2022
Source  : World Bank, World Development Indicators (series SP.POP.TOTL)
          https://databank.worldbank.org/source/world-development-indicators
x        : Year
y        : Total population (persons)

This script fits  $y = a_0 + a_1x$  by least squares using the formulas
covered in class (no np.polyfit / no library regression call), then
reports  $S_r$  (SSE),  $r^2$ , the standard error of the estimate  $s(y/x)$ , and
produces a fitted-line plot and a residual plot.
"""

import numpy as np
import matplotlib.pyplot as plt

# -----
# 1. Data (x = year, y = population in persons)
# -----
year = np.array([2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013,
                 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022])

population = np.array([88015962, 89508986, 91075184, 92699095, 94384250,
                      96337125, 98248614, 100175512, 102076336, 103767130,
                      105312992, 106735719, 108119693, 109465287, 110804683,
                      112081264, 113100950, 113964338])

n = len(year)

# -----
# 2. Least-squares formulas:
#  $a_1 = (n \cdot \text{Sum}(xy) - \text{Sum}(x) \cdot \text{Sum}(y)) / (n \cdot \text{Sum}(x^2) - (\text{Sum}(x))^2)$ 
#  $a_0 = \bar{y} - a_1 \bar{x}$ 
# -----
sum_x = np.sum(year)
sum_y = np.sum(population)
sum_xy = np.sum(year * population)
sum_x2 = np.sum(year ** 2)

a1 = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x ** 2)
a0 = (sum_y / n) - a1 * (sum_x / n)

print("Regression equation:  $y = {:.1f} + {:.1f} \cdot x$ ".format(a0, a1))
print("Slope  $a_1 = {:.4f}$  persons/year".format(a1))
print("Intercept  $a_0 = {:.4f}$  persons".format(a0))

# -----
# 3. Goodness of fit:  $S_r$  (SSE),  $S_t$ ,  $r^2$ , standard error  $s(y/x)$ 
# -----
y_pred = a0 + a1 * year
y_bar = np.mean(population)

St = np.sum((population - y_bar) ** 2) # total sum of squares
Sr = np.sum((population - y_pred) ** 2) # sum of squares of residuals (SSE)
r2 = (St - Sr) / St
syx = np.sqrt(Sr / (n - 2)) # standard error of the estimate

print("St (total sum of squares) = {:.4e}".format(St))
print("Sr (SSE, sum of squared resid.) = {:.4e}".format(Sr))
```

```

print("r^2                                = {:.6f}".format(r2))
print("s(y/x) (standard error)           = {:.2f} persons".format(syx))

# -----
# 4. Prediction for an x not in the dataset
# -----
x_new = 2027
y_new = a0 + a1 * x_new
print("\nPrediction: at x = {}, y = {:.0f} persons".format(x_new, y_new))

# -----
# 5. Plots (green / white color theme)
# -----
GREEN_DARK = "#1b5e20"
GREEN_MED  = "#43a047"
GREEN_SOFT = "#a5d6a7"
GREEN_BG   = "#f1f8f2"

plt.rcParams.update({
    "font.size": 11,
    "axes.edgecolor": GREEN_DARK,
    "axes.labelcolor": GREEN_DARK,
    "xtick.color": GREEN_DARK,
    "ytick.color": GREEN_DARK,
    "text.color": GREEN_DARK,
})

# --- Fitted line plot ---
fig, ax = plt.subplots(figsize=(7, 4.5), facecolor="white")
ax.set_facecolor(GREEN_BG)
ax.scatter(year, population / 1e6, color=GREEN_DARK, s=45, zorder=3,
           label="Observed data")
x_line = np.linspace(year.min(), x_new, 100)
ax.plot(x_line, (a0 + a1 * x_line) / 1e6, color=GREEN_MED, linewidth=2.2,
        zorder=2, label="Fitted line: y = a0 + a1x")
ax.scatter([x_new], [y_new / 1e6], color="white", edgecolor=GREEN_DARK,
           s=70, zorder=4, marker="D", label="Prediction (2027)")
ax.set_xlabel("Year")
ax.set_ylabel("Population (millions)")
ax.set_title("Philippines Population vs. Year – Least-Squares Fit",
             color=GREEN_DARK, fontweight="bold")
ax.grid(True, color=GREEN_SOFT, linewidth=0.6)
ax.legend(facecolor="white", edgecolor=GREEN_SOFT)
fig.tight_layout()
fig.savefig("fit_plot.png", dpi=200, facecolor="white")
plt.close(fig)

# --- Residual plot ---
residuals = population - y_pred
fig, ax = plt.subplots(figsize=(7, 4), facecolor="white")
ax.set_facecolor(GREEN_BG)
ax.axhline(0, color=GREEN_DARK, linewidth=1.4)
ax.scatter(year, residuals, color=GREEN_MED, s=45, edgecolor=GREEN_DARK,
           zorder=3)
ax.set_xlabel("Year")
ax.set_ylabel("Residual (persons)")
ax.set_title("Residual Plot", color=GREEN_DARK, fontweight="bold")
ax.grid(True, color=GREEN_SOFT, linewidth=0.6)
fig.tight_layout()
fig.savefig("residual_plot.png", dpi=200, facecolor="white")
plt.close(fig)

print("\nSaved fit_plot.png and residual_plot.png")

```

Part C — Regression Results

Regression equation (fitted values substituted):

$$y = -3,108,362,260.2 + 1,594,415.3 x$$

Quantity	Symbol	Value
Slope	a1	1,594,415.3 persons/year
Intercept	a0	-3,108,362,260.2 persons
Sum of squares of residuals (SSE)	Sr	8.4726×10^{12}
Total sum of squares	St	1.2401×10^{15}
Coefficient of determination	r^2	0.9932
Standard error of the estimate	s(y/x)	727,694.7 persons

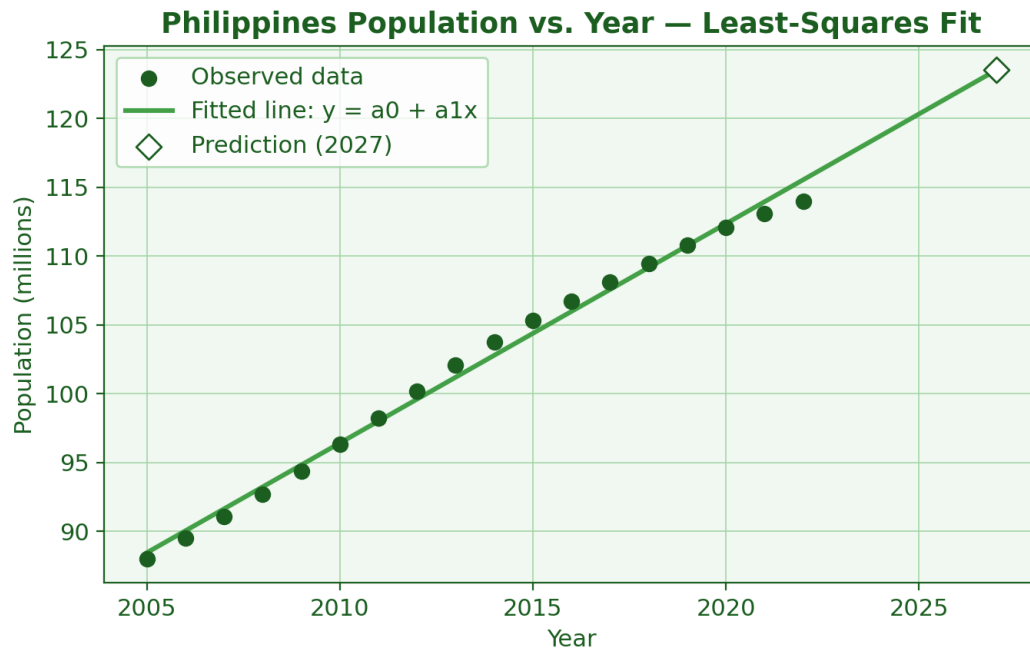


Figure 1. Observed data with the least-squares fitted line; the diamond marker shows the 2027 prediction.

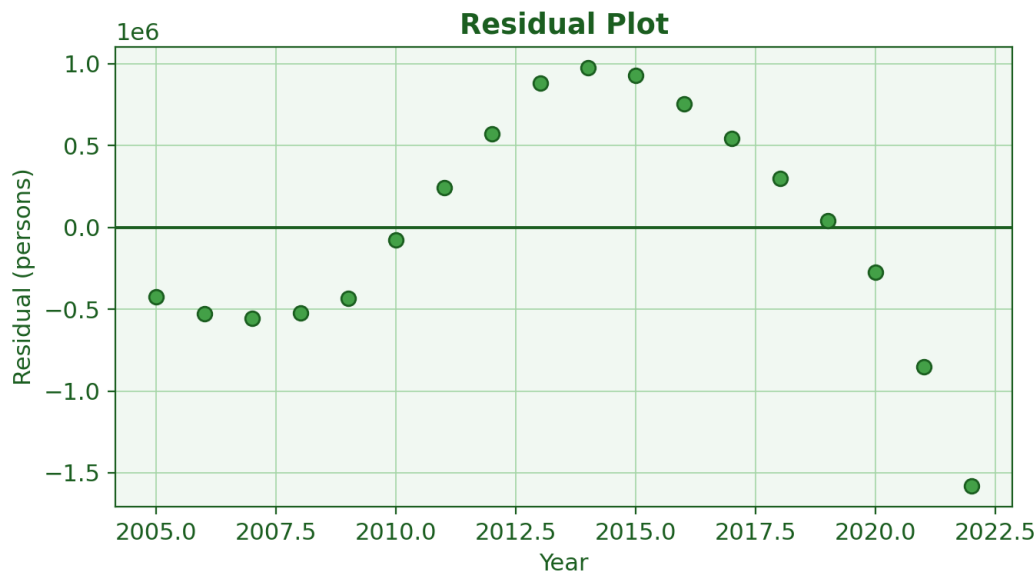


Figure 2. Residuals (observed – predicted) versus year.

Prediction

For $x = 2027$ (not in the dataset, which ends at 2022):

$$y = -3,108,362,260.2 + 1,594,415.3(2027) = 123,517,558 \text{ persons}$$

This prediction is meaningful because 2027 is only 5 years past the last observation, well within the range where short-term extrapolation of a strong linear trend ($r^2 = 0.9932$) is reasonable, and it gives a concrete, checkable estimate of the Philippines' near-future population for planning purposes (e.g., infrastructure or resource demand).

Interpretation

Slope ($a_1 = 1,594,415$ persons/year): on average, the Philippine population grew by about 1.59 million people per year over 2005–2022.

Intercept ($a_0 = -3.11 \times 10^9$): this is the y -value the line would take at year 0 and has no physical meaning by itself — it is only a mathematical artifact of extrapolating the line far outside the data range (year 0 is nowhere near 2005–2022).

Goodness of fit: $r^2 = 0.9932$ means about 99.3% of the year-to-year variation in population is explained by the linear trend, and $s(y/x) \approx 728,000$ persons is small relative to population values on the order of 10^8 , so the straight line is an excellent approximation over this period.

Residuals: the residual plot is not purely random — it shows a mild arc (negative in the earliest and latest years, positive in the middle years around 2011–2018). This systematic curvature indicates the true growth pattern is slightly nonlinear (population growth rate has been gradually slowing), so a straight line is a very good but not perfect model; a low-order polynomial or exponential model would likely fit marginally better.